

Sistema de Gestión de Emergencias Universitarias

Informe de Desarrollo e Integración (Pisos 1-3)

Backend: Joaquín Felipez Rojas & David Willy Cruz

Frontend: Nicolás Reguerin & Jhoan Calderon

18 de Junio de 2026

Índice

1. Información General	2
2. Funcionalidad Implementada	2
2.1. Descripción de la Funcionalidad	2
2.2. Objetivo	2
2.3. Requerimientos Involucrados	2
3. Desarrollo Realizado	3
3.1. Arquitectura Utilizada	3
3.2. Clases o Módulos Desarrollados	3
3.3. Capturas del Sistema Funcionando	3
4. Actividades de Debugging	4
4.1. Error 1: Clases border-border, bg-background, text-foreground no existen	4
4.2. Error 2: Conflictos al fusionar p1-frontend-base con dev	4
4.3. Error 3: Módulos de backend no encontrados tras merge	4
4.4. Error 4: Guards de ruta apuntando a la redirección anterior	4
5. Pruebas Realizadas	4
6. Conclusiones	6
7. Anexos	6
7.1. Enlace del Repositorio	6
7.2. Script de Base de Datos	6
7.3. Historial de Commits (Git Log)	8

1. Información General

- **Nombre del Proyecto:** Sistema de Gestión de Emergencias Universitarias
- **Integrantes del Equipo:**
 - **Desarrollo Backend:** Joaquín Felipez Rojas y David Willy Cruz.
 - **Desarrollo Frontend:** Nicolás y Jhoan.
- **Sprint o Fase Desarrollada:** Pisos 1 (Cimentación), 2 (Estructura - Autenticación) y 3 (Core - Gestión de Incidentes).

2. Funcionalidad Implementada

2.1. Descripción de la Funcionalidad

El sistema permite que los miembros de la comunidad universitaria reporten de manera ágil situaciones de emergencia dentro del campus (como incendios, cortes de energía, accidentes, inundaciones, problemas de seguridad u otros). Una vez reportados, estos incidentes quedan registrados con su tipo, ubicación exacta, severidad y descripción para permitir un seguimiento riguroso.

2.2. Objetivo

Proveer una plataforma centralizada y eficiente para el registro, asignación, seguimiento y auditoría de incidentes y emergencias universitarias, logrando coordinar una atención oportuna y registrar un historial fiable para futuras auditorías.

2.3. Requerimientos Involucrados

- **RF-001 (Reportar Incidente):** El usuario debe poder reportar un incidente especificando tipo, severidad, ubicación y descripción.
- **RF-002 (Ver Detalle de Incidente):** El usuario debe poder ver la información completa de un incidente.
- **RF-003 (Listar Incidentes):** El administrador debe poder listar todos los incidentes con paginación.
- **RF-008 (Autenticación):** Los usuarios deben poder iniciar sesión en el sistema.
- **RF-009 (Historial de Cambios):** El sistema debe registrar cada cambio de estado y responsable del incidente.
- **RNF-004 y RNF-005 (Seguridad):** Restricción de acceso solo a usuarios autenticados y validación de roles en endpoints críticos.
- **RN-003:** El tipo de incidente debe ser: *incendio*, *corte_energía*, *accidente*, *inundación*, *seguridad*, *otro*.
- **RN-004:** La severidad debe ser: *baja*, *media*, *alta*, *crítica*.

3. Desarrollo Realizado

3.1. Arquitectura Utilizada

La solución utiliza una arquitectura de Monorepo estructurada de la siguiente manera:

- **Backend (API RESTful):** NestJS escrito en TypeScript, modular, con inyección de dependencias. Usa **Prisma ORM** para conectarse de forma segura y tipada a una base de datos relacional.
- **Base de Datos: PostgreSQL** que reside en un contenedor Docker, garantizando un entorno reproducible y limpio.
- **Frontend (Interfaz de Usuario):** React + Vite para compilación instantánea y desarrollo ágil. Para los estilos y UI se utiliza Tailwind CSS v4, Base UI / Radix UI y componentes shadcn/ui v2. La gestión de estado asíncrono se realiza mediante TanStack Query y los formularios se manejan con React Hook Form + Zod.

3.2. Clases o Módulos Desarrollados

El sistema cuenta con las siguientes clases y entidades principales:

1. **Usuario (User):** Modelo base para almacenar credenciales (email, hash de contraseña bcrypt), estado y datos generales.
2. **Rol (Role):** Modelo para gestionar dinámicamente los roles (*admin, responder, user*).
3. **Incidente (Incident):** Entidad que almacena los detalles específicos del reporte (título, descripción, tipo, severidad, estado, ubicación).
4. **Auditoría (AuditLog):** Entidad encargada de registrar cronológicamente todas las acciones y cambios de estado efectuados sobre los incidentes.
5. **Área Campus (CampusArea):** Lugares específicos del campus que facilitan la localización geográfica del incidente.

3.3. Capturas del Sistema Funcionando

The screenshot shows a web application interface for reporting emergencies. On the left is a sidebar with navigation links: Dashboard, Incidentes, Reportar Incidente, Usuarios, and Estadísticas. The main header area displays 'Sistema de Gestión de Emergencias' and a user profile 'A. Administrador'. The central content area is titled 'Reportar Incidente' with the subtitle 'Reporte un nuevo incidente de emergencia'. Below this is a 'Formulario de Reporte' containing several input fields: 'Título del incidente' (with an example 'Ej: Incendio en edificio A'), 'Descripción' (with a placeholder 'Describe detalladamente lo sucedido...'), 'Tipo de incidente' (a dropdown menu with 'Seleccione tipo'), 'Severidad' (a dropdown menu with 'Seleccione severidad'), and 'Ubicación' (with an example 'Ej: Edificio A, piso 3, sala 301'). At the bottom of the form is a prominent black button labeled 'Reportar Incidente'.

Figura 1: Formulario de Registro y Reporte de Incidentes en React + Tailwind v4.

4. Actividades de Debugging

Durante el desarrollo e integración de las ramas en la rama central `dev`, se identificaron y resolvieron los siguientes problemas críticos:

4.1. Error 1: Clases `border-border`, `bg-background`, `text-foreground` no existen

- **Descripción:** Fallo en la compilación de PostCSS en Vite indicando que las clases de diseño de shadcn no se encontraban.
- **Causa:** El setup de shadcn se inicializó asumiendo Tailwind CSS v4, pero la aplicación usaba Tailwind CSS v3.
- **Solución:** Se instaló Tailwind v4 y su plugin oficial para Vite (`@tailwindcss/vite`), se actualizó `index.css` y se eliminaron los archivos de configuración obsoletos v3 (`tailwind.config.js` y `postcss.config.js`).

4.2. Error 2: Conflictos al fusionar `p1-frontend-base` con `dev`

- **Descripción:** Múltiples conflictos en `package.json`, `App.tsx` e `index.css`.
- **Causa:** Dos ramas de tareas paralelas del Piso 1 colisionaron al modificar el mismo árbol de directorios del frontend.
- **Solución:** Se resolvieron manualmente los archivos conservando la base tecnológica moderna (Tailwind v4 y Base UI), re-instalando las dependencias necesarias.

4.3. Error 3: Módulos de backend no encontrados tras merge

- **Descripción:** Errores del tipo `Cannot find module '@nestjs/jwt'`.
- **Causa:** Las dependencias requeridas para la autenticación se instalaron localmente pero no se commitearon ni registraron en el lockfile general.

- **Solución:** Se ejecutó `pnpm install` para reconstruir y sincronizar el archivo de bloqueo de pnpm de forma correcta.

4.4. Error 4: Guards de ruta apuntando a la redirección anterior

- **Descripción:** Los guards redirigían al usuario no autenticado a `/login` en lugar de `/auth`.
- **Causa:** Al unificar las pantallas de autenticación en `/auth` con Tabs, las redirecciones estáticas de los guards no fueron actualizadas.
- **Solución:** Se modificó la dirección en `ProtectedRoute.tsx`, `AdminRoute.tsx` y `ResponsableRoute.tsx` para usar el endpoint correcto.

5. Pruebas Realizadas

Se validó la estabilidad del monorepo mediante pruebas locales exhaustivas:

- **Build check del Monorepo:** Ejecución de `pnpm build` garantizando que tanto la aplicación frontend como backend transpilen a JavaScript nativo sin errores de empaquetado.
- **Verificación de Tipos estáticos:** Ejecución de `pnpm check-types` garantizando integridad estricta en todas las interfaces, esquemas Zod y controladores.
- **Flujo de Autenticación:** Pruebas de generación y firma de tokens JWT junto con la decodificación por parte del backend y almacenamiento en cliente mediante cookies/localStorage.

6. Conclusiones

- **Lecciones Aprendidas:** La importancia de tener un flujo de Git bien planificado y realizar sincronizaciones continuas con la rama `dev` para prevenir conflictos masivos en dependencias y configuración. Además, la adopción de Tailwind CSS v4 requiere actualizar la configuración clásica a la nueva sintaxis basada en CSS nativo.
- **Mejoras Futuras:** Implementación de comunicación bidireccional por WebSockets para notificaciones en tiempo real, e integración de geolocalización real mediante mapas interactivos.

7. Anexos

7.1. Enlace del Repositorio

El código fuente completo del proyecto y su historial de desarrollo se encuentra disponible en:

- **Repositorio Principal (GitHub):** <https://github.com/Joaquinf87/hackaton>

7.2. Script de Base de Datos

El siguiente script en SQL representa la base de datos PostgreSQL generada por las migraciones de Prisma para el monorepo:

```

1  -- CreateTable
2  CREATE TABLE "users" (
3      "id" TEXT NOT NULL,
4      "name" TEXT NOT NULL,
5      "email" TEXT NOT NULL,
6      "password_hash" TEXT NOT NULL,
7      "phone" TEXT,
8      "is_active" BOOLEAN NOT NULL DEFAULT true,
9      "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
10     "updated_at" TIMESTAMP(3) NOT NULL,
11     CONSTRAINT "users_pkey" PRIMARY KEY ("id")
12 );
13
14 -- CreateTable
15 CREATE TABLE "roles" (
16     "id" TEXT NOT NULL,
17     "name" TEXT NOT NULL,
18     "description" TEXT,
19     "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
20     CONSTRAINT "roles_pkey" PRIMARY KEY ("id")
21 );
22
23 -- CreateTable
24 CREATE TABLE "user_roles" (
25     "user_id" TEXT NOT NULL,
26     "role_id" TEXT NOT NULL,
27     CONSTRAINT "user_roles_pkey" PRIMARY KEY ("user_id","role_id")
28 );
29
30 -- CreateTable
31 CREATE TABLE "campus_areas" (
32     "id" TEXT NOT NULL,

```

```
33     "name" TEXT NOT NULL,
34     "description" TEXT,
35     "location" TEXT,
36     "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
37     CONSTRAINT "campus_areas_pkey" PRIMARY KEY ("id")
38 );
39
40 -- CreateTable
41 CREATE TABLE "incidents" (
42     "id" TEXT NOT NULL,
43     "title" TEXT NOT NULL,
44     "description" TEXT NOT NULL,
45     "type" TEXT NOT NULL,
46     "severity" TEXT NOT NULL,
47     "status" TEXT NOT NULL DEFAULT 'reported',
48     "location" TEXT NOT NULL,
49     "reported_by_id" TEXT NOT NULL,
50     "assigned_to_id" TEXT,
51     "area_id" TEXT,
52     "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
53     "updated_at" TIMESTAMP(3) NOT NULL,
54     CONSTRAINT "incidents_pkey" PRIMARY KEY ("id")
55 );
56
57 -- CreateTable
58 CREATE TABLE "audit_logs" (
59     "id" TEXT NOT NULL,
60     "incident_id" TEXT NOT NULL,
61     "user_id" TEXT NOT NULL,
62     "action" TEXT NOT NULL,
63     "old_status" TEXT,
64     "new_status" TEXT,
65     "comment" TEXT,
66     "created_at" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
67     CONSTRAINT "audit_logs_pkey" PRIMARY KEY ("id")
68 );
69
70 -- CreateIndex
71 CREATE UNIQUE INDEX "users_email_key" ON "users"("email");
72 CREATE UNIQUE INDEX "roles_name_key" ON "roles"("name");
73 CREATE UNIQUE INDEX "campus_areas_name_key" ON "campus_areas"("name");
```

7.3. Historial de Commits (Git Log)

A continuación se detallan los últimos commits registrados en la rama de desarrollo:

```
1 d96a685 Merge remote-tracking branch 'origin/feature/p3-reportar-incidente' into dev
2 7502b2d Merge remote-tracking branch 'origin/feature/p3-incidents-api' into dev
3 99d4ad2 Merge remote-tracking branch 'origin/p3-auditoria-servicio' into dev
4 44638a9 feat: creacion de CRUD de incidentes en NestJs
5 4bf432a feat(p3): crear formulario para reportar incidente con validacion de zod
6 084b75f p2(fix): Fix de rutas protegidas
7 b9be72a Feat: implementacion servicio de auditoria
8 fe4381b Merge pull request #9 from Joaquinfr87/feature/p2-auth-guard-frontend
9 3ca0943 feat(p2): implementar guards de rutas por rol y redireccion de
    autenticacion
10 6ab90bb feat(p2): implement roles service, guard, and decorator
11 393ce12 p2(fix): se unio la rama auth-pages con la rama auth-hook, se resolvio
    los conflictos
12 e49c20e Merge feature/p2-auth-pages into dev
13 634846a Merge pull request #8 from Joaquinfr87/feature/p2-auth-hook
14 458839c p2(feat): se anadio de hook personalizado useAuth con tanstack query
15 a13017c feat(p2): unificar login y register con tabs
```